GUÍA DEL ALUMNO

Pruebas automatizadas documentación OpenAPI e integración inicial con React

EC3 F2 A11 API de gestión de tareas

Dato	Descripción
Asignatura	Programación Backend con Frameworks
Punto de partida	task-api-express al finalizar EC3 F2 A10
Producto	API probada y documentada con un cliente React mínimo
Herramientas	Visual Studio Code, Node.js, TypeScript, Express 5, PNPM, Mongoose 8.24.4, MongoDB Atlas, Vitest, SuperTest, OpenAPI, Swagger UI, React, Vite, Postman, Git y GitHub

Meta de aprendizaje. Convertir la API protegida de A10 en un servicio que pueda comprobarse automáticamente, entenderse mediante una especificación ejecutable y consumirse desde una aplicación React sin duplicar reglas del backend.

Universidad Estatal de Sonora

1 Propósito

En esta actividad comprobarás que la API mantiene su contrato después de cada cambio. Incorporarás pruebas HTTP automatizadas con Vitest y SuperTest, publicarás una especificación OpenAPI 3.1 mediante Swagger UI y crearás un cliente React mínimo que iniciará sesión, validará la identidad con GET /api/auth/me y consultará tareas protegidas.

La integración será deliberadamente pequeña. No construirás todavía el frontend completo ni desplegarás la solución. La actividad 12 quedará reservada para completar la integración, preparar ambientes de producción, desplegar y liberar la aplicación.

2 Resultado de aprendizaje

Al finalizar la actividad serás capaz de:

- distinguir pruebas automatizadas aisladas de pruebas manuales contra MongoDB Atlas;
- probar rutas, middleware, validación y autorización sin depender de una base de datos externa;
- describir rutas, parámetros, cuerpos, esquemas, seguridad y respuestas mediante OpenAPI 3.1;
- servir Swagger UI sin debilitar las cabeceras del resto de la API;
- obtener la identidad asociada a un JWT mediante GET /api/auth/me;
- consumir una API protegida desde React con fetch y el encabezado Authorization;
- interpretar errores CORS, 401, 403, 422 y respuestas no JSON;
- ejecutar una verificación repetible del backend y del cliente antes de confirmar cambios.

3 Punto de partida y conocimientos previos

Trabaja sobre el mismo repositorio task-api-express terminado en A10. Deben funcionar el registro, el inicio de sesión, Bearer JWT, los roles user y admin, el CRUD persistente, la paginación, los filtros, la búsqueda, el ordenamiento, Helmet, CORS, el límite de autenticación, AppError y el manejador central de errores.

Componente heredado	Estado al iniciar A11	Decisión
POST /register y /login	Públicos y limitados	Se conservan y se prueban.
JWT y roles	Protegen /api/tasks	Se reutilizan en pruebas, Swagger y React.
CRUD de tareas	Persistente y protegido	No se reescribe.
CORS	Permite http://localhost:5173	Se usa para el cliente Vite.
Manejador de errores	Contrato uniforme	Se documenta y se verifica.
Mongoose	Versión 8.24.4	Se conserva sin migración mayor.

Antes de iniciar debes saber ejecutar scripts de PNPM, interpretar solicitudes HTTP, usar async y await, leer tipos de TypeScript, crear una colección de Postman y abrir dos terminales. No necesitas conocer pruebas automatizadas ni OpenAPI previamente.

4 Producto que se construirá

El backend publicará GET /api-docs.json y una interfaz Swagger en /api-docs. También incorporará GET /api/auth/me para que un cliente pueda validar el token y recuperar datos actuales del usuario sin guardar el perfil como fuente definitiva en el navegador.

Producto	Comprobación principal
Suite automatizada	pnpm test ejecuta seis casos y comprueba varias respuestas HTTP.
Verificación completa	pnpm verify valida tipos, pruebas y builds de backend y cliente.
OpenAPI	GET /api-docs.json devuelve openapi 3.1.0 y define bearerAuth.
Swagger UI	http://localhost:3000/api-docs permite explorar y probar rutas.
Identidad	GET /api/auth/me devuelve el usuario del JWT o 401.
Cliente React	Login, consulta /me, lista tareas, muestra errores y cierra la sesión local.

5 Conceptos fundamentales antes de programar

Concepto	Aplicación en esta guía
Prueba automatizada	Código que prepara un escenario, ejecuta la operación y compara el resultado esperado.
Doble de prueba	Sustituto controlado de un servicio. Permite probar HTTP sin escribir en Atlas.
Prueba de regresión	Comprueba que una función ya existente continúa funcionando después del cambio.
OpenAPI	Contrato legible por personas y herramientas que describe operaciones, seguridad y esquemas.
Swagger UI	Interfaz web que representa la especificación y permite enviar solicitudes.
Contrato de API	Acuerdo observable sobre método, ruta, datos, códigos y estructura de respuesta.
CORS	Política del navegador. Debe permitir el origen exacto del cliente Vite.
sessionStorage	Almacena el token solo durante la pestaña. Sigue siendo accesible a JavaScript y no elimina el riesgo XSS.
Separación de responsabilidades	React presenta y solicita; Express valida y aplica reglas; MongoDB persiste.

Pirámide aplicada. Las pruebas de esta guía verifican la frontera HTTP con servicios simulados, por lo que son rápidas y repetibles. Las pruebas con Postman comprueban el sistema real con Atlas. La prueba desde React confirma finalmente el comportamiento del navegador, incluido CORS.

Un doble de servicio no demuestra que Atlas esté disponible. Su propósito es detectar fallas de rutas, middleware, cuerpos, tokens, roles y respuestas. Debes conservar también una comprobación manual con tu clúster.

6 Decisiones de continuidad y correcciones de contexto

A10 aplicó authRateLimiter al router completo porque solo existían register y login. A11 agrega GET /api/auth/me, que no es un intento de credenciales. Por ello el limitador se mueve a cada POST público; así /me conserva autenticación sin consumir el límite de acceso.

El desafío opcional de A10 proponía /api/auth/me, pero A11 no puede asumir que todos los estudiantes lo resolvieron. La ruta se incorpora formalmente con una implementación común. Si ya la creaste, compara tu solución y sustituye los archivos por la versión de esta guía para mantener el contrato del grupo.

El cliente guarda temporalmente el access token en sessionStorage porque A9 y A10 no implementaron cookies HttpOnly ni refresh tokens. Esta decisión permite observar el flujo Bearer, pero no se presenta como la solución más segura para producción.

7 Arquitectura y estructura resultante

Estructura relevante

```
task-api-express/
├── package.json
├── pnpm-workspace.yaml
├── tsconfig.test.json
├── vitest.config.ts
├── src/
│   ├── app.ts
│   ├── docs/
│   │   └── openapi.ts
│   ├── controllers/auth.controller.ts
│   ├── repositories/user.repository.ts
│   ├── routes/auth.routes.ts
│   ├── services/auth.service.ts
│   └── middlewares/security.middleware.ts
├── tests/
│   ├── setup-env.ts
│   └── api.test.ts
└── client/
    ├── .env.example
    ├── package.json
    ├── index.html
    ├── tsconfig.json
    ├── tsconfig.app.json
    ├── tsconfig.node.json
    ├── vite.config.ts
    └── src/
        ├── api.ts
        ├── App.tsx
        ├── main.tsx
        ├── styles.css
        └── types.ts
```

Flujo integrado. React envía credenciales a login, recibe el access token, lo guarda en la pestaña y lo adjunta como Bearer. Express verifica el token, /me recupera el usuario actual y /api/tasks entrega los datos. OpenAPI documenta ese contrato y Vitest lo comprueba sin abrir un puerto ni tocar Atlas.

8 Desarrollo de la aplicación

Sigue el orden indicado. Todo bloque marcado como código completo sustituye el archivo señalado. Los archivos del CRUD que no aparecen permanecen sin cambios. Guarda cada archivo antes de ejecutar comandos.

Parte I Preparación del proyecto

Paso 1. Crear la rama de A11 y verificar A10

Objetivo. Separar el trabajo y comprobar que la base no contiene errores previos.

La nueva actividad debe comenzar desde el commit final de A10. Ejecuta la verificación disponible en tu proyecto; si todavía no existe verify, utiliza check y build.

Ruta. Raíz del repositorio task-api-express; no se modifica un archivo.

Comando

```
git status
git switch -c ec3-a11-calidad-integracion
pnpm install
pnpm check
pnpm build
```

Resultado esperado. La rama está activa y TypeScript compila la solución de A10.

Posibles errores y corrección.

- Si la rama ya existe, usa git switch ec3-a11-calidad-integracion.
- Si A10 falla, corrige o recupera su entrega antes de agregar dependencias.
- No continúes con archivos .env, tokens o URI preparados para commit.

Paso 2. Instalar documentación y herramientas de prueba

Objetivo. Agregar dependencias exactas y scripts reproducibles.

swagger-ui-express publica la interfaz. Vitest ejecuta los casos y SuperTest envía solicitudes directamente a app sin usar app.listen. Los tipos se instalan como dependencias de desarrollo. Los scripts client separan claramente los dos proyectos.

Ruta. package.json

Código completo

```
{
  "name": "task-api-express",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/server.ts",
    "check": "tsc --noEmit && tsc -p tsconfig.test.json --noEmit",
    "build": "tsc",
    "start": "node dist/server.js",
    "role:set": "tsx src/scripts/set-user-role.ts",
    "test": "vitest run",
    "test:watch": "vitest",
    "client:dev": "pnpm --dir client dev",
    "client:check": "pnpm --dir client check",
    "client:build": "pnpm --dir client build",
    "verify": "pnpm check && pnpm test && pnpm build && pnpm client:check && pnpm client:build"
  },
  "dependencies": {
    "bcryptjs": "3.0.3",
    "cors": "2.8.6",
    "dotenv": "17.2.2",
    "express": "5.1.0",
    "express-rate-limit": "8.7.1",
    "helmet": "8.3.0",
    "jose": "6.2.12",
    "mongoose": "8.24.4",
    "swagger-ui-express": "5.0.1"
  },
  "devDependencies": {
    "@types/cors": "2.8.19",
    "@types/express": "5.0.3",
    "@types/node": "24.3.1",
    "@types/supertest": "7.2.1",
    "@types/swagger-ui-express": "4.1.8",
    "supertest": "7.3.1",
    "tsx": "4.20.5",
    "typescript": "5.9.2",
    "vitest": "5.0.4"
  },
  "engines": {
    "node": ">=20.19.0"
  }
}
```

Partes importantes.

- Mongoose 8.24.4, Express 5.1.0 y el resto de A10 se conservan.
- test ejecuta una sola vez; test:watch vuelve a ejecutar al guardar.
- verify detiene la cadena en el primer error y construye backend y cliente.

Comando

```
pnpm install
```

Resultado esperado. PNPM actualiza pnpm-lock.yaml e instala Swagger UI, Vitest y SuperTest.

Posibles errores y corrección.

- Si aparece un aviso de scripts de @scarf/scarf, completa el paso 3 y repite pnpm install.
- No instales paquetes con npm: el proyecto usa un solo pnpm-lock.yaml.
- No actualices Mongoose a 9 para esta actividad; no aporta una función necesaria.

Paso 3. Convertir backend y cliente en un workspace

Objetivo. Permitir que PNPM instale y ejecute ambos paquetes de manera coherente.

El workspace incluye la raíz y client. esbuild debe ejecutarse para Vite. El script opcional de Scarf se desactiva de forma explícita; Swagger UI funciona sin ese script de instalación.

Ruta. pnpm-workspace.yaml

Código completo

```
packages:
  - .
  - client

allowBuilds:
  '@scarf/scarf': false
  esbuild: true
```

Partes importantes.

- La decisión false evita ejecutar un script opcional que no es necesario para el curso.
- client se instalará dentro del mismo lockfile, pero conserva su propio package.json.
- No agregues node_modules ni dist al repositorio.

Comando

```
pnpm install
```

Resultado esperado. La salida indica Scope: all 2 workspace projects.

Posibles errores y corrección.

- Si client no se instala, confirma que aparezca exactamente como segundo paquete.
- Si esbuild queda bloqueado, conserva allowBuilds con esbuild: true.
- No escribas set this to true or false: esa frase es una indicación interactiva, no un valor YAML válido.

Parte II Identidad actual y documentación OpenAPI

Paso 4. Consultar un usuario por su identificador

Objetivo. Agregar al repositorio la operación mínima que necesita /me.

El repositorio mantiene el acceso a Mongoose y el mapeo de errores. findById respeta select: false del passwordHash, por lo que la consulta no recupera el hash.

Ruta. src/repositories/user.repository.ts

Código completo

```
import mongoose, { type HydratedDocument } from 'mongoose';
import { AppError } from '../errors/app-error.js';
import { UserModel, type UserPersistence, type UserRole } from '../models/user.js';

export type UserDocument = HydratedDocument<UserPersistence>;

export interface CreateUserRecord {
  name: string;
  email: string;
  passwordHash: string;
  role: UserRole;
}

const isDuplicateKeyError = (error: unknown): boolean =>
  typeof error === 'object'
  && error !== null
  && 'code' in error
  && error.code === 11_000;

const mapPersistenceError = (error: unknown): AppError => {
  if (error instanceof AppError) return error;
  if (isDuplicateKeyError(error)) {
    return new AppError(
      'Ya existe una cuenta con ese correo electrónico.',
      409,
      'USER_EMAIL_EXISTS'
    );
  }
  if (error instanceof mongoose.Error.ValidationError) {
    const details = Object.values(error.errors).map((item) => ({
      field: item.path,
      message: item.message
    }));
    return new AppError(
      'El usuario no cumple las reglas del modelo.',
      422,
      'PERSISTENCE_VALIDATION_ERROR',
      details
    );
  }
  return new AppError(
    'No fue posible acceder al almacenamiento de usuarios.',
    503,
    'DATABASE_UNAVAILABLE'
  );
};

const execute = async <T>(operation: () => Promise<T>): Promise<T> => {
  try {
    return await operation();
  } catch (error: unknown) {
    throw mapPersistenceError(error);
  }
};

export const findUserDocumentByEmail = async (
  email: string,
  includePasswordHash = false
): Promise<UserDocument | null> => execute(async () => {
  const query = UserModel.findOne({ email });
  if (includePasswordHash) query.select('+passwordHash');
  return query;
});

export const findUserDocumentById = async (
  id: string
```

```
): Promise<UserDocument | null> => execute(async () => UserModel.findById(id));

export const createUserDocument = async (
  data: CreateUserRecord
): Promise<UserDocument> => execute(async () => UserModel.create(data));

export const setUserRoleByEmail = async (
  email: string,
  role: UserRole
): Promise<UserDocument | null> => execute(async () =>
  UserModel.findOneAndUpdate(
    { email },
    { $set: { role } },
    { new: true, runValidators: true }
  )
);
```

Partes importantes.

- findUserDocumentByEmail continúa usándose en registro y login.
- findUserDocumentById usa execute para conservar los errores de persistencia uniformes.
- No uses lean aquí porque el servicio ya trabaja con UserDocument.

Comando

```
pnpm check
```

Resultado esperado. El repositorio exporta la nueva función sin modificar el modelo.

Posibles errores y corrección.

- Si findById no existe, revisa que UserModel se importe desde models/user.js.
- No agregues +passwordHash: /me nunca necesita la contraseña.
- Con NodeNext, conserva .js en importaciones relativas.

Paso 5. Recuperar la identidad actual desde el servicio

Objetivo. Convertir el id del token en un usuario público actualizado.

El token demuestra que la firma era válida, pero la cuenta pudo eliminarse después. El servicio consulta el usuario y devuelve 401 si ya no existe. toUser garantiza que passwordHash nunca forma parte de la respuesta.

Ruta. src/services/auth.service.ts

Código completo

```
import { env } from '../config/env.js';
import { AppError } from '../errors/app-error.js';
import type { User } from '../models/user.js';
import {
  createUserDocument,
  findUserDocumentById,
  findUserDocumentByEmail,
  type UserDocument
} from '../repositories/user.repository.js';
import type {
  LoginInput,
  LoginResult,
  RegisterInput,
  RegisterResult
} from '../types/auth.js';
import { hashPassword, verifyPassword } from './password.service.js';
import { createAccessToken } from './token.service.js';

const DUMMY_PASSWORD_HASH =
  '$2b$12$ryC/a3t1mI7fqF6j4w3Z.OpHQerUsIQUPQYXvd4WSgCRuikv6IF82';

const toUser = (document: UserDocument): User => ({
  id: document._id.toString(),
  name: document.name,
```

```
  email: document.email,
  role: document.role,
  createdAt: document.createdAt,
  updatedAt: document.updatedAt
});

export const registerUser = async (
  input: RegisterInput
): Promise<RegisterResult> => {
  const email = input.email.trim().toLowerCase();
  const existing = await findUserDocumentByEmail(email);
  if (existing) {
    throw new AppError(
      'Ya existe una cuenta con ese correo electrónico.',
      409,
      'USER_EMAIL_EXISTS'
    );
  }
  const passwordHash = await hashPassword(input.password);
  const document = await createUserDocument({
    name: input.name,
    email,
    passwordHash,
    role: 'user'
  });
  return { user: toUser(document) };
};

export const loginUser = async (input: LoginInput): Promise<LoginResult> => {
  const email = input.email.trim().toLowerCase();
  const document = await findUserDocumentByEmail(email, true);
  const passwordHash = document?.passwordHash ?? DUMMY_PASSWORD_HASH;
  const validPassword = await verifyPassword(input.password, passwordHash);
  if (!document || !validPassword) {
    throw new AppError('El correo o la contraseña son incorrectos.', 401, 'INVALID_CREDENTIALS');
  }
  const user = toUser(document);
  return {
    user,
    accessToken: await createAccessToken(user),
    tokenType: 'Bearer',
    expiresIn: env.JWT_EXPIRES_IN
  };
};

export const getAuthenticatedUser = async (userId: string): Promise<User> => {
  const document = await findUserDocumentById(userId);
  if (!document) {
    throw new AppError(
      'La cuenta asociada al token ya no está disponible.',
      401,
      'AUTHENTICATED_USER_NOT_FOUND'
    );
  }
  return toUser(document);
};
```

Partes importantes.

- AUTHENTICATED_USER_NOT_FOUND indica que el token ya no representa una cuenta disponible.
- La regla pertenece al servicio; el controlador no consulta Mongoose.
- La función conserva fechas y rol actuales del documento.

Resultado esperado. getAuthenticatedUser devuelve User o lanza AppError 401.

Posibles errores y corrección.

- Si TypeScript no encuentra findUserDocumentById, verifica su exportación e importación.
- No devuelvas el documento Mongoose directamente porque podría ampliar el contrato.
- No conviertas la ausencia en 404: para este flujo la autenticación ya no es válida.

Paso 6. Crear GET /api/auth/me y limitar solo las credenciales

Objetivo. Exponer la identidad autenticada sin castigar la consulta con el límite de login.

El controlador recibe userId desde res.locals.auth. El router aplica authenticate a /me y coloca authRateLimiter directamente en register y login. Esta sustitución corrige el alcance que tenía authRouter.use en A10.

Ruta. src/controllers/auth.controller.ts y src/routes/auth.routes.ts

Partes importantes.

- El controlador no recibe el id desde params ni body; confía únicamente en el JWT verificado.
- Express 5 propaga la promesa rechazada de getAuthenticatedUser al errorHandler.
- register y login permanecen públicos; /me exige Bearer JWT.

Comando

```
pnpm check
```

Resultado esperado. GET /api/auth/me queda protegido y los dos POST conservan el límite.

Posibles errores y corrección.

- Si /me devuelve 429 durante uso normal, authRateLimiter probablemente sigue montado con authRouter.use.
- Si auth no está tipado, conserva src/types/express.d.ts de A10.
- No aceptes userId enviado por el navegador; permitiría consultar otra identidad.

src/controllers/auth.controller.ts

Código completo

```
import type { Request, Response } from 'express';
import {
  getAuthenticatedUser,
  loginUser,
  registerUser
} from '../services/auth.service.js';

export const postRegister = async (_req: Request, res: Response): Promise<void> => {
  res.status(201).json({ data: await registerUser(res.locals.registerInput) });
};

export const postLogin = async (_req: Request, res: Response): Promise<void> => {
  res.status(200).json({ data: await loginUser(res.locals.loginInput) });
};

export const getMe = async (_req: Request, res: Response): Promise<void> => {
  const auth = res.locals.auth;
  if (!auth) {
    throw new Error('La ruta /me requiere el middleware authenticate.');
  }
  res.status(200).json({
    data: { user: await getAuthenticatedUser(auth.userId) }
  });
};
```

src/routes/auth.routes.ts

Código completo

```
import { Router } from 'express';
import { getMe, postLogin, postRegister } from '../controllers/auth.controller.js';
import { authenticate } from '../middlewares/authenticate.middleware.js';
import { requireJson } from '../middlewares/require-json.middleware.js';
import { authRateLimiter } from '../middlewares/security.middleware.js';
import { validateLogin, validateRegister } from '../middlewares/validate-auth.middleware.js';

export const authRouter = Router();
authRouter.post(
  '/register',
  authRateLimiter,
  requireJson,
  validateRegister,
  postRegister
);
authRouter.post('/login', authRateLimiter, requireJson, validateLogin, postLogin);
authRouter.get('/me', authenticate, getMe);
```

Paso 7. Preparar cabeceras compatibles con Swagger UI

Objetivo. Mantener cabeceras de seguridad sin impedir que cargue la documentación.

Swagger UI necesita ejecutar recursos y estilos propios. En sus rutas se aplican las cabeceras de Helmet con Content Security Policy desactivada; el resto de la API continúa usando securityHeaders completo.

Ruta. src/middlewares/security.middleware.ts

Código completo

```
import cors, { type CorsOptions } from 'cors';
import { rateLimit } from 'express-rate-limit';
import helmet from 'helmet';
import { env } from '../config/env.js';
import { AppError } from '../errors/app-error.js';

const corsOptions: CorsOptions = {
  origin(origin, callback) {
    if (!origin || origin === env.CORS_ORIGIN) {
      callback(null, true);
      return;
    }
    callback(null, false);
  },
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  exposedHeaders: ['X-Request-Id', 'RateLimit', 'RateLimit-Policy'],
  maxAge: 600
};

export const securityHeaders = helmet();
export const documentationSecurityHeaders = helmet({
  contentSecurityPolicy: false
});
export const corsPolicy = cors(corsOptions);

export const authRateLimiter = rateLimit({
  windowMs: env.AUTH_RATE_LIMIT_WINDOW_MS,
  limit: env.AUTH_RATE_LIMIT_MAX,
  standardHeaders: 'draft-8',
  legacyHeaders: false,
  handler(_req, _res, next) {
    next(new AppError(
      'Demasiados intentos de autenticación. Intenta más tarde.',
      429,
      'AUTH_RATE_LIMIT_EXCEEDED'
    ));
  }
});
```

Partes importantes.

- La excepción está limitada a /api-docs y /api-docs.json desde app.ts.
- No se desactiva Helmet globalmente.
- CORS sigue permitiendo el origen configurado del cliente React.

Resultado esperado. Existe documentationSecurityHeaders además de securityHeaders.

Posibles errores y corrección.

- Si Swagger aparece en blanco y la consola informa CSP, confirma que app.ts use el middleware especial antes del global.
- No agregues unsafe-inline a toda la API solo para corregir la documentación.
- Helmet no sustituye HTTPS.

Paso 8. Definir el contrato OpenAPI completo

Objetivo. Describir rutas, seguridad, parámetros, esquemas y errores con una sola fuente versionada.

El documento se expresa como un objeto TypeScript serializable. Documenta salud, registro, login, /me y todo el CRUD. bearerAuth indica a Swagger UI cómo construir Authorization: Bearer. Los componentes reutilizables evitan repetir esquemas de error.

Ruta. src/docs/openapi.ts

Código completo

```
export const openApiDocument: Record<string, unknown> = {
  openapi: '3.1.0',
  info: {
    title: 'Task API Express',
    version: '1.0.0',
    description:
      'API REST para gestionar tareas con autenticación JWT y autorización por roles.'
  },
  servers: [
    {
      url: 'http://localhost:3000',
      description: 'Entorno local'
    }
  ],
  tags: [
    { name: 'Health', description: 'Comprobación del servicio' },
    { name: 'Auth', description: 'Registro, acceso e identidad autenticada' },
    { name: 'Tasks', description: 'CRUD protegido de tareas' }
  ],
  paths: {
    '/health': {
      get: {
        tags: ['Health'],
        summary: 'Comprobar que la API responde',
        responses: {
          '200': {
            description: 'Servicio disponible',
            content: {
              'application/json': {
                schema: {
                  type: 'object',
                  required: ['data'],
                  properties: {
                    data: {
                      type: 'object',
                      required: ['status'],
                      properties: { status: { type: 'string', example: 'up' } }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
```

```
  },
  '/health/database': {
    get: {
      tags: ['Health'],
      summary: 'Comprobar el estado de la conexión con MongoDB',
      responses: {
        '200': {
          description: 'Base de datos conectada',
          content: {
            'application/json': {
              schema: {
                type: 'object',
                required: ['data'],
                properties: {
                  data: {
                    type: 'object',
                    required: ['status'],
                    properties: { status: { type: 'string', const: 'up' } }
                  }
                }
              }
            }
          }
        },
        '503': {
          description: 'Base de datos no disponible',
          content: {
            'application/json': {
              schema: {
                type: 'object',
                required: ['data'],
                properties: {
                  data: {
                    type: 'object',
                    required: ['status'],
                    properties: { status: { type: 'string', const: 'down' } }
                  }
                }
              }
            }
          }
        }
      }
    }
  },
  '/api/auth/register': {
    post: {
      tags: ['Auth'],
      summary: 'Registrar una cuenta de usuario',
      requestBody: {
        required: true,
        content: {
          'application/json': {
            schema: { $ref: '#/components/schemas/RegisterInput' }
          }
        }
      },
      responses: {
        '201': {
          description: 'Cuenta creada',
          content: {
            'application/json': {
              schema: { $ref: '#/components/schemas/UserEnvelope' }
            }
          }
        },
        '409': { $ref: '#/components/responses/Conflict' },
        '415': { $ref: '#/components/responses/UnsupportedMediaType' },
        '422': { $ref: '#/components/responses/ValidationError' },
        '429': { $ref: '#/components/responses/RateLimited' }
      }
    }
  },
  '/api/auth/login': {
    post: {
      tags: ['Auth'],
      summary: 'Iniciar sesión y obtener un access token',
      requestBody: {
        required: true,
        content: {
          'application/json': {
            schema: { $ref: '#/components/schemas/LoginInput' }
          }
```

```
        }
      },
      responses: {
        '200': {
          description: 'Credenciales correctas',
          content: {
            'application/json': {
              schema: { $ref: '#/components/schemas/LoginEnvelope' }
            }
          }
        },
        '401': { $ref: '#/components/responses/Unauthorized' },
        '415': { $ref: '#/components/responses/UnsupportedMediaType' },
        '422': { $ref: '#/components/responses/ValidationError' },
        '429': { $ref: '#/components/responses/RateLimited' }
      }
    }
  },
  '/api/auth/me': {
    get: {
      tags: ['Auth'],
      summary: 'Obtener el usuario asociado al token',
      security: [{ bearerAuth: [] }],
      responses: {
        '200': {
          description: 'Identidad autenticada',
          content: {
            'application/json': {
              schema: { $ref: '#/components/schemas/UserEnvelope' }
            }
          }
        },
        '401': { $ref: '#/components/responses/Unauthorized' }
      }
    }
  },
  '/api/tasks': {
    get: {
      tags: ['Tasks'],
      summary: 'Listar tareas con paginación, filtros y ordenamiento',
      security: [{ bearerAuth: [] }],
      parameters: [
        { $ref: '#/components/parameters/Page' },
        { $ref: '#/components/parameters/Limit' },
        { $ref: '#/components/parameters/Status' },
        { $ref: '#/components/parameters/Search' },
        { $ref: '#/components/parameters/SortBy' },
        { $ref: '#/components/parameters/Order' }
      ],
      responses: {
        '200': {
          description: 'Lista paginada',
          content: {
            'application/json': {
              schema: { $ref: '#/components/schemas/TaskListResult' }
            }
          }
        },
        '401': { $ref: '#/components/responses/Unauthorized' },
        '422': { $ref: '#/components/responses/ValidationError' }
      }
    },
    post: {
      tags: ['Tasks'],
      summary: 'Crear una tarea',
      security: [{ bearerAuth: [] }],
      requestBody: {
        required: true,
        content: {
          'application/json': {
            schema: { $ref: '#/components/schemas/CreateTaskInput' }
          }
        }
      },
      responses: {
        '201': {
          description: 'Tarea creada',
          content: {
            'application/json': {
              schema: { $ref: '#/components/schemas/TaskEnvelope' }
            }
          }
        },
```

```
      '401': { $ref: '#/components/responses/Unauthorized' },
      '415': { $ref: '#/components/responses/UnsupportedMediaType' },
      '422': { $ref: '#/components/responses/ValidationError' }
    }
  }
},
'/api/tasks/{id}': {
  parameters: [{ $ref: '#/components/parameters/TaskId' }],
  get: {
    tags: ['Tasks'],
    summary: 'Consultar una tarea',
    security: [{ bearerAuth: [] }],
    responses: {
      '200': {
        description: 'Tarea encontrada',
        content: {
          'application/json': {
            schema: { $ref: '#/components/schemas/TaskEnvelope' }
          }
        }
      },
      '401': { $ref: '#/components/responses/Unauthorized' },
      '404': { $ref: '#/components/responses/NotFound' },
      '422': { $ref: '#/components/responses/ValidationError' }
    }
  },
  patch: {
    tags: ['Tasks'],
    summary: 'Modificar título o estado de una tarea',
    security: [{ bearerAuth: [] }],
    requestBody: {
      required: true,
      content: {
        'application/json': {
          schema: { $ref: '#/components/schemas/UpdateTaskInput' }
        }
      }
    },
    responses: {
      '200': {
        description: 'Tarea modificada',
        content: {
          'application/json': {
            schema: { $ref: '#/components/schemas/TaskEnvelope' }
          }
        }
      },
      '401': { $ref: '#/components/responses/Unauthorized' },
      '404': { $ref: '#/components/responses/NotFound' },
      '415': { $ref: '#/components/responses/UnsupportedMediaType' },
      '422': { $ref: '#/components/responses/ValidationError' }
    }
  },
  delete: {
    tags: ['Tasks'],
    summary: 'Eliminar una tarea como administrador',
    security: [{ bearerAuth: [] }],
    responses: {
      '204': { description: 'Tarea eliminada' },
      '401': { $ref: '#/components/responses/Unauthorized' },
      '403': { $ref: '#/components/responses/Forbidden' },
      '404': { $ref: '#/components/responses/NotFound' },
      '422': { $ref: '#/components/responses/ValidationError' }
    }
  }
},
'/api/tasks/{id}/complete': {
  patch: {
    tags: ['Tasks'],
    summary: 'Marcar una tarea como completada',
    security: [{ bearerAuth: [] }],
    parameters: [{ $ref: '#/components/parameters/TaskId' }],
    responses: {
      '200': {
        description: 'Tarea completada',
        content: {
          'application/json': {
            schema: { $ref: '#/components/schemas/TaskEnvelope' }
          }
        }
      },
      '401': { $ref: '#/components/responses/Unauthorized' },
      '404': { $ref: '#/components/responses/NotFound' },
```

```
        '422': { $ref: '#/components/responses/ValidationError' }
      }
    }
  }
},
components: {
  securitySchemes: {
    bearerAuth: {
      type: 'http',
      scheme: 'bearer',
      bearerFormat: 'JWT'
    }
  },
  parameters: {
    TaskId: {
      name: 'id',
      in: 'path',
      required: true,
      schema: { type: 'string', pattern: '^[a-fA-F0-9]{24}$' }
    },
    Page: {
      name: 'page',
      in: 'query',
      schema: { type: 'integer', minimum: 1, default: 1 }
    },
    Limit: {
      name: 'limit',
      in: 'query',
      schema: { type: 'integer', minimum: 1, maximum: 100, default: 10 }
    },
    Status: {
      name: 'status',
      in: 'query',
      schema: { type: 'string', enum: ['pending', 'completed'] }
    },
    Search: {
      name: 'search',
      in: 'query',
      schema: { type: 'string', maxLength: 80 }
    },
    SortBy: {
      name: 'sortBy',
      in: 'query',
      schema: {
        type: 'string',
        enum: ['title', 'status', 'createdAt', 'updatedAt'],
        default: 'createdAt'
      }
    },
    Order: {
      name: 'order',
      in: 'query',
      schema: { type: 'string', enum: ['asc', 'desc'], default: 'asc' }
    }
  },
  schemas: {
    User: {
      type: 'object',
      required: ['id', 'name', 'email', 'role', 'createdAt', 'updatedAt'],
      properties: {
        id: { type: 'string', example: '67112bfe47963b2be12c43a1' },
        name: { type: 'string', example: 'Ana Torres' },
        email: { type: 'string', format: 'email', example: 'ana@example.com' },
        role: { type: 'string', enum: ['user', 'admin'] },
        createdAt: { type: 'string', format: 'date-time' },
        updatedAt: { type: 'string', format: 'date-time' }
      }
    },
    Task: {
      type: 'object',
      required: ['id', 'title', 'status', 'createdAt', 'updatedAt'],
      properties: {
        id: { type: 'string', example: '67112bfe47963b2be12c43a2' },
        title: { type: 'string', example: 'Documentar la API' },
        status: { type: 'string', enum: ['pending', 'completed'] },
        createdAt: { type: 'string', format: 'date-time' },
        updatedAt: { type: 'string', format: 'date-time' }
      }
    },
    RegisterInput: {
      type: 'object',
      required: ['name', 'email', 'password'],
      additionalProperties: false,
```

```
    properties: {
      name: { type: 'string', minLength: 2, maxLength: 80 },
      email: { type: 'string', format: 'email', maxLength: 254 },
      password: { type: 'string', minLength: 15, format: 'password' }
    }
  },
  LoginInput: {
    type: 'object',
    required: ['email', 'password'],
    additionalProperties: false,
    properties: {
      email: { type: 'string', format: 'email' },
      password: { type: 'string', format: 'password' }
    }
  },
  CreateTaskInput: {
    type: 'object',
    required: ['title'],
    additionalProperties: false,
    properties: {
      title: { type: 'string', minLength: 1, maxLength: 120 }
    }
  },
  UpdateTaskInput: {
    type: 'object',
    minProperties: 1,
    additionalProperties: false,
    properties: {
      title: { type: 'string', minLength: 1, maxLength: 120 },
      status: { type: 'string', enum: ['pending', 'completed'] }
    }
  },
  ErrorDetail: {
    type: 'object',
    additionalProperties: true,
    properties: {
      field: { type: 'string' },
      message: { type: 'string' }
    }
  },
  ApiError: {
    type: 'object',
    required: ['code', 'message'],
    properties: {
      code: { type: 'string', example: 'VALIDATION_ERROR' },
      message: { type: 'string', example: 'La solicitud contiene datos inválidos.' },
      details: {
        type: 'array',
        items: { $ref: '#/components/schemas/ErrorDetail' }
      },
      requestId: { type: 'string', format: 'uuid' }
    }
  },
  ErrorEnvelope: {
    type: 'object',
    required: ['error'],
    properties: { error: { $ref: '#/components/schemas/ApiError' } }
  },
  UserEnvelope: {
    type: 'object',
    required: ['data'],
    properties: {
      data: {
        type: 'object',
        required: ['user'],
        properties: { user: { $ref: '#/components/schemas/User' } }
      }
    }
  },
  LoginEnvelope: {
    type: 'object',
    required: ['data'],
    properties: {
      data: {
        type: 'object',
        required: ['user', 'accessToken', 'tokenType', 'expiresIn'],
        properties: {
          user: { $ref: '#/components/schemas/User' },
          accessToken: { type: 'string' },
          tokenType: { type: 'string', const: 'Bearer' },
          expiresIn: { type: 'string', example: '1h' }
        }
      }
```

```
    }
   },
   TaskEnvelope: {
    type: 'object',
    required: ['data'],
    properties: { data: { $ref: '#/components/schemas/Task' } }
   },
   PaginationMeta: {
    type: 'object',
    required: [
     'page',
     'limit',
     'total',
     'totalPages',
     'hasNextPage',
     'hasPreviousPage'
    ],
    properties: {
     page: { type: 'integer' },
     limit: { type: 'integer' },
     total: { type: 'integer' },
     totalPages: { type: 'integer' },
     hasNextPage: { type: 'boolean' },
     hasPreviousPage: { type: 'boolean' }
    }
   },
   TaskListResult: {
    type: 'object',
    required: ['data', 'meta'],
    properties: {
     data: { type: 'array', items: { $ref: '#/components/schemas/Task' } },
     meta: { $ref: '#/components/schemas/PaginationMeta' }
    }
   }
  },
  responses: {
   Unauthorized: {
    description: 'Token ausente, inválido, vencido o sin cuenta asociada',
    content: {
     'application/json': {
      schema: { $ref: '#/components/schemas/ErrorEnvelope' }
     }
    }
   },
   Forbidden: {
    description: 'El rol autenticado no tiene permiso',
    content: {
     'application/json': {
      schema: { $ref: '#/components/schemas/ErrorEnvelope' }
     }
    }
   },
   NotFound: {
    description: 'Recurso inexistente',
    content: {
     'application/json': {
      schema: { $ref: '#/components/schemas/ErrorEnvelope' }
     }
    }
   },
   Conflict: {
    description: 'Conflicto con un recurso existente',
    content: {
     'application/json': {
      schema: { $ref: '#/components/schemas/ErrorEnvelope' }
     }
    }
   },
   UnsupportedMediaType: {
    description: 'El cuerpo no utiliza application/json',
    content: {
     'application/json': {
      schema: { $ref: '#/components/schemas/ErrorEnvelope' }
     }
    }
   },
   ValidationError: {
    description: 'Parámetros, cuerpo o identificador inválidos',
    content: {
     'application/json': {
      schema: { $ref: '#/components/schemas/ErrorEnvelope' }
     }
    }
```

```
    },
    RateLimited: {
      description: 'Se superó el límite temporal de autenticación',
      content: {
        'application/json': {
          schema: { $ref: '#/components/schemas/ErrorEnvelope' }
        }
      }
    }
  }
 }
};
```

Partes importantes.

- OpenAPI describe el comportamiento; no ejecuta la validación del servidor.
- Los códigos documentados deben coincidir con middleware, controladores y AppError.
- passwordHash, JWT_SECRET y MONGODB_URI no aparecen en ejemplos ni esquemas.

Resultado esperado. openApiDocument contiene OpenAPI 3.1.0, securitySchemes.bearerAuth y todas las rutas actuales.

Posibles errores y corrección.

- Si Swagger no muestra una operación, revisa llaves y el nombre exacto de paths.
- Si Try it out devuelve 401, usa Authorize e introduce solo el token, sin escribir Bearer.
- No documentes una respuesta 200 cuando el código produce 201 o 204.

Paso 9. Publicar JSON y Swagger UI desde Express

Objetivo. Hacer que la documentación viaje con la misma versión de la API.

requestContext se ejecuta primero. Después se publican /api-docs.json y /api-docs con cabeceras específicas. Las rutas funcionales conservan Helmet global, CORS, límite de JSON, notFound y errorHandler.

Ruta. src/app.ts

Código completo

```
import express from 'express';
import swaggerUi from 'swagger-ui-express';
import { openApiDocument } from './docs/openapi.js';
import { errorHandler } from './middlewares/error.middleware.js';
import { notFound } from './middlewares/not-found.middleware.js';
import { requestContext } from './middlewares/request-context.middleware.js';
import {
  corsPolicy,
  documentationSecurityHeaders,
  securityHeaders
} from './middlewares/security.middleware.js';
import { authRouter } from './routes/auth.routes.js';
import { healthRouter } from './routes/health.routes.js';
import { taskRouter } from './routes/task.routes.js';

export const app = express();
app.disable('x-powered-by');
app.use(requestContext);
app.get(
  '/api-docs.json',
  documentationSecurityHeaders,
  (_req, res): void => {
    res.status(200).json(openApiDocument);
  }
);
app.use(
  '/api-docs',
  documentationSecurityHeaders,
  swaggerUi.serve,
  swaggerUi.setup(openApiDocument, {
    customSiteTitle: 'Task API Express'
  })
);
app.use(securityHeaders);
app.use(corsPolicy);
app.use(express.json({ limit: '20kb' }));
app.use('/health', healthRouter);
app.use('/api/auth', authRouter);
app.use('/api/tasks', taskRouter);
app.use(notFound);
app.use(errorHandler);
```

Partes importantes.

- /api-docs.json permite que otras herramientas importen la especificación.
- /api-docs/ sirve la interfaz y sus recursos estáticos.
- La documentación se registra antes de notFound y errorHandler.

Comando

```
pnpm check
pnpm build
```

Resultado esperado. TypeScript compila y dist incluye app.js y docs/openapi.js.

Posibles errores y corrección.

- Si faltan tipos, confirma @types/swagger-ui-express en devDependencies.
- Si Node no encuentra ./docs/openapi.js, la importación de TypeScript debe terminar en .js.
- Si /api-docs devuelve 404, revisa el orden antes de notFound.

Parte III Pruebas automatizadas

Paso 10. Crear una configuración TypeScript para pruebas

Objetivo. Revisar también el código de Vitest sin mezclarlo con dist del backend.

tsconfig.json conserva rootDir en src. El archivo adicional amplía la raíz para incluir tests y vitest.config.ts, agrega los tipos de Vitest y usa un directorio que no se genera durante noEmit.

Ruta. tsconfig.test.json

Código completo

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "rootDir": ".",
    "outDir": "dist-test",
    "types": ["node", "vitest/globals"]
  },
  "include": ["src/**/*.ts", "tests/**/*.ts", "vitest.config.ts"],
  "exclude": ["node_modules", "dist", "dist-test"]
}
```

Partes importantes.

- El script check ejecuta primero la aplicación y después esta configuración.
- strict y noUncheckedIndexedAccess se heredan.
- Las pruebas no forman parte del build de producción.

Resultado esperado. pnpm check puede validar src, tests y configuración.

Posibles errores y corrección.

- Si aparece rootDir, confirma que rootDir sea punto en este archivo.
- Si describe o expect no tienen tipo, agrega vitest/globals en types.
- No elimines strict para ocultar errores de las pruebas.

Paso 11. Configurar Vitest y un ambiente seguro

Objetivo. Preparar variables no sensibles antes de importar la aplicación.

env.ts se ejecuta al importar módulos, por lo que setupFiles define valores de prueba. La URI apunta a localhost, pero los servicios se reemplazan y no se abre una conexión. El máximo de intentos se eleva para que la suite no se interfiera a sí misma.

Ruta. vitest.config.ts y tests/setup-env.ts

Partes importantes.

- fileParallelism false evita interferencias entre estados globales de middleware.
- restoreMocks y clearMocks restablecen dobles entre casos.
- La clave de prueba no se reutiliza en desarrollo ni producción.

Resultado esperado. Vitest conoce el ambiente node y carga variables antes de app.

Posibles errores y corrección.

- Si falta una variable al ejecutar test, confirma setupFiles y el nombre exacto.
- No uses la URI o JWT_SECRET reales en pruebas automatizadas del repositorio.
- No llames connectDatabase desde api.test.ts.

vitest.config.ts

Código completo

```
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    environment: 'node',
    setupFiles: ['./tests/setup-env.ts'],
    fileParallelism: false,
    clearMocks: true,
    restoreMocks: true
  }
});
```

tests/setup-env.ts

Código completo

```
process.env.PORT = '3000';
process.env.MONGODB_URI = 'mongodb://127.0.0.1:27017/task_api_test';
process.env.MONGODB_DB_NAME = 'task_api_test';
process.env.JWT_SECRET = '0123456789abcdef0123456789abcdef';
process.env.JWT_EXPIRES_IN = '1h';
process.env.JWT_ISSUER = 'task-api-express';
process.env.JWT_AUDIENCE = 'task-api-client';
process.env.CORS_ORIGIN = 'http://localhost:5173';
process.env.AUTH_RATE_LIMIT_WINDOW_MS = '900000';
process.env.AUTH_RATE_LIMIT_MAX = '100';
```

Paso 12. Probar la frontera HTTP con dobles de servicio

Objetivo. Comprobar rutas y middleware de forma rápida sin modificar Atlas.

vi.mock sustituye auth.service y task.service antes de importar app. Los tokens siguen siendo JWT reales creados por token.service, de modo que authenticate y requireRole se prueban. SuperTest envía solicitudes a Express directamente, sin abrir un puerto.

Ruta. tests/api.test.ts

Código completo

```
import request from 'supertest';
import { beforeEach, describe, expect, it, vi } from 'vitest';
import type { Task } from '../src/models/task.js';
import type { User } from '../src/models/user.js';
import type {
  LoginInput,
  LoginResult,
  RegisterInput,
  RegisterResult
} from '../src/types/auth.js';
import type { TaskListQuery, TaskListResult } from '../src/types/task-query.js';

const authServiceDoubles = vi.hoisted(() => ({
  registerUser: vi.fn<(input: RegisterInput) => Promise<RegisterResult>>(),
  loginUser: vi.fn<(input: LoginInput) => Promise<LoginResult>>(),
  getAuthenticatedUser: vi.fn<(userId: string) => Promise<User>>()
}));

const taskServiceDoubles = vi.hoisted(() => ({
  listTasks: vi.fn<(query: TaskListQuery) => Promise<TaskListResult>>(),
  findTaskById: vi.fn<(id: string) => Promise<Task>>(),
```

```
  createTask: vi.fn<(title: string) => Promise<Task>>(),
  updateTask: vi.fn<(id: string, changes: object) => Promise<Task>>(),
  completeTask: vi.fn<(id: string) => Promise<Task>>(),
  deleteTask: vi.fn<(id: string) => Promise<void>>()
}));

vi.mock('../src/services/auth.service.js', () => authServiceDoubles);
vi.mock('../src/services/task.service.js', () => taskServiceDoubles);

import { app } from '../src/app.js';
import { createAccessToken } from '../src/services/token.service.js';

const user: User = {
  id: '67112bfe47963b2be12c43a1',
  name: 'Ana Torres',
  email: 'ana.torres@example.com',
  role: 'user',
  createdAt: new Date('2026-10-01T12:00:00.000Z'),
  updatedAt: new Date('2026-10-01T12:00:00.000Z')
};

const admin: User = { ...user, id: '67112bfe47963b2be12c43a2', role: 'admin' };

const task: Task = {
  id: '67112bfe47963b2be12c43b1',
  title: 'Ejecutar pruebas automatizadas',
  status: 'pending',
  createdAt: new Date('2026-10-02T12:00:00.000Z'),
  updatedAt: new Date('2026-10-02T12:00:00.000Z')
};

let userToken = '';
let adminToken = '';

beforeEach(async () => {
  vi.clearAllMocks();
  userToken = await createAccessToken(user);
  adminToken = await createAccessToken(admin);

  authServiceDoubles.registerUser.mockResolvedValue({ user });
  authServiceDoubles.loginUser.mockResolvedValue({
    user,
    accessToken: userToken,
    tokenType: 'Bearer',
    expiresIn: '1h'
  });
  authServiceDoubles.getAuthenticatedUser.mockResolvedValue(user);

  taskServiceDoubles.listTasks.mockResolvedValue({
    data: [task],
    meta: {
      page: 1,
      limit: 10,
      total: 1,
      totalPages: 1,
      hasNextPage: false,
      hasPreviousPage: false
    }
  });
  taskServiceDoubles.findTaskById.mockResolvedValue(task);
  taskServiceDoubles.createTask.mockResolvedValue(task);
  taskServiceDoubles.updateTask.mockResolvedValue(task);
  taskServiceDoubles.completeTask.mockResolvedValue({ ...task, status: 'completed' });
  taskServiceDoubles.deleteTask.mockResolvedValue();
});

describe('documentación y controles HTTP', () => {
  it('publica OpenAPI, Swagger UI y encabezados de seguridad', async () => {
    const health = await request(app).get('/health').expect(200);
    expect(health.body.data.status).toBe('up');
    expect(health.headers['x-powered-by']).toBeUndefined();
    expect(health.headers['x-content-type-options']).toBe('nosniff');
    expect(typeof health.headers['x-request-id']).toBe('string');

    const specification = await request(app).get('/api-docs.json').expect(200);
    expect(specification.body.openapi).toBe('3.1.0');
    expect(specification.body.paths['/api/auth/me']).toBeDefined();
    expect(specification.body.components.securitySchemes.bearerAuth.scheme).toBe('bearer');

    const swagger = await request(app).get('/api-docs/').expect(200);
    expect(swagger.headers['content-type']).toContain('text/html');
    expect(swagger.text).toContain('<title>Task API Express</title>');
    expect(swagger.text).toContain('id="swagger-ui"');
```

```
  });
});

describe('autenticación e identidad', () => {
  it('registra, autentica y consulta /me sin exponer el hash', async () => {
    const registration = await request(app)
      .post('/api/auth/register')
      .send({
        name: 'Ana Torres',
        email: 'ana.torres@example.com',
        password: 'Frase segura para clase 2026'
      })
      .expect(201);
    expect(registration.body.data.user.passwordHash).toBeUndefined();

    const login = await request(app)
      .post('/api/auth/login')
      .send({
        email: 'ana.torres@example.com',
        password: 'Frase segura para clase 2026'
      })
      .expect(200);
    expect(login.body.data.tokenType).toBe('Bearer');

    const me = await request(app)
      .get('/api/auth/me')
      .set('Authorization', `Bearer ${userToken}`)
      .expect(200);
    expect(me.body.data.user.email).toBe(user.email);
    expect(authServiceDoubles.getAuthenticatedUser).toHaveBeenCalledWith(user.id);
  });

  it('rechaza /me y las tareas cuando falta el token', async () => {
    const me = await request(app).get('/api/auth/me').expect(401);
    expect(me.body.error.code).toBe('AUTHENTICATION_REQUIRED');

    const taskList = await request(app).get('/api/tasks').expect(401);
    expect(taskList.body.error.code).toBe('AUTHENTICATION_REQUIRED');
  });
});

describe('CRUD protegido y autorización', () => {
  it('envía al servicio la consulta validada y crea una tarea', async () => {
    const list = await request(app)
      .get('/api/tasks?page=1&limit=10&status=pending&search=pruebas&sortBy=title&order=asc')
      .set('Authorization', `Bearer ${userToken}`)
      .expect(200);
    expect(list.body.data).toHaveLength(1);
    expect(taskServiceDoubles.listTasks).toHaveBeenCalledWith({
      page: 1,
      limit: 10,
      status: 'pending',
      search: 'pruebas',
      sortBy: 'title',
      order: 'asc'
    });

    const created = await request(app)
      .post('/api/tasks')
      .set('Authorization', `Bearer ${userToken}`)
      .send({ title: task.title })
      .expect(201);
    expect(created.body.data.title).toBe(task.title);
    expect(taskServiceDoubles.createTask).toHaveBeenCalledWith(task.title);
  });

  it('mantiene 403 para user y 204 para admin al eliminar', async () => {
    const forbidden = await request(app)
      .delete(`/api/tasks/${task.id}`)
      .set('Authorization', `Bearer ${userToken}`)
      .expect(403);
    expect(forbidden.body.error.code).toBe('FORBIDDEN');
    expect(taskServiceDoubles.deleteTask).not.toHaveBeenCalled();

    await request(app)
      .delete(`/api/tasks/${task.id}`)
      .set('Authorization', `Bearer ${adminToken}`)
      .expect(204);
    expect(taskServiceDoubles.deleteTask).toHaveBeenCalledWith(task.id);
  });
});

describe('regresión y respuestas negativas', () => {
```

```
  it('conserva validación, CORS y contrato uniforme de error', async () => {
    const invalid = await request(app)
      .get('/api/tasks?page=0')
      .set('Authorization', `Bearer ${userToken}`)
      .expect(422);
    expect(invalid.body.error.code).toBe('QUERY_VALIDATION_ERROR');
    expect(typeof invalid.body.error.requestId).toBe('string');
    expect(taskServiceDoubles.listTasks).not.toHaveBeenCalled();

    const allowedOrigin = await request(app)
      .get('/health')
      .set('Origin', 'http://localhost:5173')
      .expect(200);
    expect(allowedOrigin.headers['access-control-allow-origin']).toBe(
      'http://localhost:5173'
    );

    const blockedOrigin = await request(app)
      .get('/health')
      .set('Origin', 'https://no-autorizado.example')
      .expect(200);
    expect(blockedOrigin.headers['access-control-allow-origin']).toBeUndefined();
  });
});
```

Partes importantes.

- La suite cubre documentación, seguridad, identidad, token ausente, validación, CORS y roles.
- Los servicios simulados aíslan HTTP; no demuestran persistencia real.
- Cada caso prepara resultados conocidos en beforeEach.

Comando

```
pnpm test
```

Resultado esperado. Vitest informa 1 archivo aprobado y 6 pruebas aprobadas.

Posibles errores y corrección.

- Si un mock no se aplica, confirma que vi.mock aparezca antes de importar app.
- Si una prueba de admin devuelve 403, verifica que el token se cree con role admin.
- Si el límite devuelve 429, revisa AUTH_RATE_LIMIT_MAX del ambiente de prueba.

Parte IV Cliente React mínimo

Paso 13. Crear el paquete del cliente y su variable pública

Objetivo. Preparar React y Vite sin mezclar secretos del backend.

El cliente es un segundo paquete dentro del workspace. VITE_API_URL es visible en el navegador y solo contiene la URL pública. Nunca copies MONGODB_URI, JWT_SECRET ni contraseñas al prefijo VITE_.

Ruta. client/package.json y client/.env.example

Partes importantes.

- React y React DOM se instalan como dependencias del cliente.
- Vite, TypeScript, plugin-react y tipos se usan en desarrollo.
- Node 20.19 o superior coincide con la base del proyecto.

Comando

```
cp client/.env.example client/.env
pnpm install
```

Resultado esperado. PNPM instala ambos paquetes y client/.env permanece ignorado.

Posibles errores y corrección.

- En Windows PowerShell, usa Copy-Item client/.env.example client/.env.
- No agregues secretos a VITE_API_URL; todo VITE_ termina en el bundle.
- Si aparece otro lockfile dentro de client, elimínalo de forma segura y usa el de la raíz.

client/package.json

Código completo

```
{
  "name": "task-client-react",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "check": "tsc -b --pretty false",
    "build": "tsc -b && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "19.3.0",
    "react-dom": "19.3.0"
  },
  "devDependencies": {
    "@types/react": "19.3.0",
    "@types/react-dom": "19.3.0",
    "@vitejs/plugin-react": "6.1.2",
    "typescript": "5.9.2",
    "vite": "8.3.4"
  },
  "engines": {
    "node": ">=20.19.0"
  }
}
```

client/.env.example

Código completo

```
VITE_API_URL=http://localhost:3000
```

.gitignore que se conserva en la raíz

Estas reglas también protegen client/.env porque el patrón .env se aplica en cualquier nivel del repositorio. Conserva el archivo .env.example como plantilla sin secretos.

Código completo

```
node_modules/
dist/
.env
.env.*
!.env.example
*.log
.DS_Store
```

Paso 14. Configurar TypeScript y Vite para React

Objetivo. Compilar JSX con tipos estrictos y usar el puerto permitido por CORS.

El proyecto referencia una configuración del navegador y otra para vite.config.ts. strictPort evita que Vite cambie silenciosamente a 5174, lo que rompería CORS_ORIGIN.

Ruta. client/tsconfig.json, client/tsconfig.app.json, client/tsconfig.node.json y client/vite.config.ts

Partes importantes.

- moduleResolution Bundler corresponde al entorno Vite, no al backend NodeNext.
- noUncheckedIndexedAccess mantiene el criterio estricto del curso.
- El puerto 5173 debe coincidir con CORS_ORIGIN del backend.

Resultado esperado. pnpm client:check valida los archivos del navegador y Vite.

Posibles errores y corrección.

- Si 5173 está ocupado, detén el proceso; no cambies de puerto sin actualizar CORS_ORIGIN.
- No copies tsconfig NodeNext del backend al cliente porque son entornos diferentes.
- Si react() no se resuelve, ejecuta pnpm install desde la raíz.

client/tsconfig.json

Código completo

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
```

client/tsconfig.app.json

Código completo

```
{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "allowJs": false,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "forceConsistentCasingInFileNames": true,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "types": ["vite/client"]
  },
  "include": ["src"]
}
```

client/tsconfig.node.json

Código completo

```
{
  "compilerOptions": {
    "composite": true,
    "skipLibCheck": true,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "allowImportingTsExtensions": true,
    "strict": true,
    "noEmit": true
  },
  "include": ["vite.config.ts"]
}
```

client/vite.config.ts

Código completo

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: true
  }
});
```

Paso 15. Crear el documento HTML y los tipos del contrato

Objetivo. Definir el punto de montaje y representar respuestas sin any.

index.html contiene el único elemento raíz. types.ts refleja los esquemas de OpenAPI para User, Task, login, lista paginada y error. Estos tipos ayudan durante el desarrollo, pero la API sigue siendo la fuente real en tiempo de ejecución.

Ruta. client/index.html y client/src/types.ts

Partes importantes.

- Las fechas llegan como string JSON aunque el backend use Date internamente.
- ApiEnvelope representa respuestas con data; TaskListResponse agrega meta.
- details usa unknown[] en lugar de any[].

Resultado esperado. El cliente dispone de contratos explícitos para las respuestas que utiliza.

Posibles errores y corrección.

- Si createRoot no encuentra root, revisa el id en index.html.
- No declares fechas como Date en el navegador sin transformarlas.
- Si cambia OpenAPI, actualiza estos tipos y las pruebas del cliente.

client/index.html

Código completo

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Cliente React para Task API Express" />
    <title>Task API Client</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

client/src/types.ts

Código completo

```
export type UserRole = 'user' | 'admin';
export type TaskStatus = 'pending' | 'completed';

export interface User {
  id: string;
  name: string;
  email: string;
  role: UserRole;
  createdAt: string;
  updatedAt: string;
}

export interface Task {
  id: string;
  title: string;
  status: TaskStatus;
  createdAt: string;
  updatedAt: string;
}

export interface PaginationMeta {
  page: number;
  limit: number;
  total: number;
  totalPages: number;
  hasNextPage: boolean;
  hasPreviousPage: boolean;
}

export interface LoginData {
  user: User;
  accessToken: string;
  tokenType: 'Bearer';
  expiresIn: string;
}

export interface ApiEnvelope<T> {
  data: T;
}

export interface TaskListResponse {
  data: Task[];
  meta: PaginationMeta;
}

export interface ApiErrorData {
  code: string;
  message: string;
  details?: unknown[];
  requestId?: string;
}
```

Paso 16. Centralizar las solicitudes con fetch

Objetivo. Evitar repetir URL, encabezados y manejo de errores.

apiRequest construye Headers, agrega Accept, Content-Type cuando existe body y Authorization cuando recibe token. Después valida la respuesta, interpreta JSON y convierte el contrato de error de Express en ApiHttpError.

Ruta. client/src/api.ts

Código completo

```
import type { ApiErrorData } from './types.js';

const API_URL = (import.meta.env.VITE_API_URL ?? 'http://localhost:3000').replace(
  /\/$/,
  ''
);

export class ApiHttpError extends Error {
  constructor(
    public readonly status: number,
    public readonly code: string,
    message: string,
    public readonly requestId?: string
  ) {
    super(message);
    this.name = 'ApiHttpError';
  }
}

const isErrorEnvelope = (value: unknown): value is { error: ApiErrorData } => {
  if (typeof value !== 'object' || value === null || !('error' in value)) return false;
  const error = value.error;
  return typeof error === 'object'
    && error !== null
    && 'code' in error
    && typeof error.code === 'string'
    && 'message' in error
    && typeof error.message === 'string';
};

export const apiRequest = async <T>(
  path: string,
  init: RequestInit = {},
  accessToken?: string
): Promise<T> => {
  const headers = new Headers(init.headers);
  headers.set('Accept', 'application/json');
  if (init.body !== undefined) headers.set('Content-Type', 'application/json');
  if (accessToken) headers.set('Authorization', `Bearer ${accessToken}`);

  const response = await fetch(`${API_URL}${path}`, { ...init, headers });
  const text = await response.text();
  let payload: unknown;
  if (text) {
    try {
      payload = JSON.parse(text) as unknown;
    } catch {
      throw new ApiHttpError(
        response.status,
        'INVALID_SERVER_RESPONSE',
        'La API devolvió una respuesta que no es JSON.'
      );
    }
  }

  if (!response.ok) {
    if (isErrorEnvelope(payload)) {
      throw new ApiHttpError(
        response.status,
        payload.error.code,
        payload.error.message,
        payload.error.requestId
      );
    }
    throw new ApiHttpError(
      response.status,
      'HTTP_ERROR',
      `La solicitud terminó con el código ${response.status}.`
```

```
    );
  }

  return payload as T;
};
```

Partes importantes.

- La función no agrega un token vacío.
- response.ok reconoce cualquier código 2xx.
- INVALID_SERVER_RESPONSE distingue HTML o texto inesperado de un error API normal.

Resultado esperado. Todos los componentes pueden consumir la API mediante una sola función genérica.

Posibles errores y corrección.

- Failed to fetch suele indicar API apagada, URL incorrecta o CORS.
- Si la respuesta contiene HTML, confirma que VITE_API_URL apunte al backend y no a Vite.
- No registres accessToken en console.log.

Paso 17. Implementar login sesión e identidad en React

Objetivo. Demostrar el flujo completo de una ruta pública y dos rutas protegidas.

App mantiene formulario, token, usuario, tareas, mensaje y estado de carga. Después del login guarda el access token en sessionStorage. Un efecto consulta /me y /api/tasks en paralelo. Un 401 limpia la sesión local.

Ruta. client/src/App.tsx

Código completo

```
import { useCallback, useEffect, useState, type FormEvent } from 'react';
import { ApiHttpError, apiRequest } from './api.js';
import type {
  ApiEnvelope,
  LoginData,
  Task,
  TaskListResponse,
  User
} from './types.js';

const TOKEN_KEY = 'task-api.access-token';

const messageFrom = (error: unknown): string => {
  if (error instanceof ApiHttpError) {
    const reference = error.requestId ? ` Referencia: ${error.requestId}.` : '';
    return `${error.message}${reference}`;
  }
  return 'No fue posible comunicarse con la API.';
};

export default function App() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [token, setToken] = useState<string | null>(() =>
    sessionStorage.getItem(TOKEN_KEY)
  );
  const [user, setUser] = useState<User | null>(null);
  const [tasks, setTasks] = useState<Task[]>([]);
  const [message, setMessage] = useState('Inicia sesión para consultar tus tareas.');
  const [busy, setBusy] = useState(false);

  const clearSession = useCallback((): void => {
    sessionStorage.removeItem(TOKEN_KEY);
    setToken(null);
    setUser(null);
    setTasks([]);
  }, []);

  const loadProtectedData = useCallback(async (accessToken: string): Promise<void> => {
    try {
      setBusy(true);
```

```
    const [meResponse, taskResponse] = await Promise.all([
      apiRequest<ApiEnvelope<{ user: User }>>('/api/auth/me', {}, accessToken),
      apiRequest<TaskListResponse>('/api/tasks?limit=10&sortBy=createdAt&order=desc', {}, accessToken)
    ]);
    setUser(meResponse.data.user);
    setTasks(taskResponse.data);
    setMessage(`Se recuperaron ${taskResponse.meta.total} tareas.`);
  } catch (error: unknown) {
    if (error instanceof ApiHttpError && error.status === 401) clearSession();
    setMessage(messageFrom(error));
  } finally {
    setBusy(false);
  }
}, [clearSession]);

useEffect(() => {
  if (token) void loadProtectedData(token);
}, [loadProtectedData, token]);

const handleLogin = async (event: FormEvent<HTMLFormElement>): Promise<void> => {
  event.preventDefault();
  setBusy(true);
  setMessage('Comprobando credenciales...');
  try {
    const response = await apiRequest<ApiEnvelope<LoginData>>('/api/auth/login', {
      method: 'POST',
      body: JSON.stringify({ email, password })
    });
    sessionStorage.setItem(TOKEN_KEY, response.data.accessToken);
    setToken(response.data.accessToken);
    setPassword('');
  } catch (error: unknown) {
    setMessage(messageFrom(error));
    setBusy(false);
  }
};

const handleLogout = (): void => {
  clearSession();
  setMessage('La sesión local se eliminó.');
};

return (
  <main className="page-shell">
    <header className="page-header">
      <div>
        <p className="eyebrow">EC3 F2 A11</p>
        <h1>Task API Client</h1>
        <p>Integración inicial de React con una API protegida por JWT.</p>
      </div>
      {user && <button onClick={handleLogout}>Cerrar sesión</button>}
    </header>

    {!token ? (
      <section className="panel login-panel" aria-labelledby="login-title">
        <h2 id="login-title">Iniciar sesión</h2>
        <form onSubmit={(event) => void handleLogin(event)}>
          <label htmlFor="email">Correo electrónico</label>
          <input
            id="email"
            type="email"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
            autoComplete="username"
            required
          />
          <label htmlFor="password">Contraseña</label>
          <input
            id="password"
            type="password"
            value={password}
            onChange={(event) => setPassword(event.target.value)}
            autoComplete="current-password"
            required
          />
          <button type="submit" disabled={busy}>
            {busy ? 'Ingresando...' : 'Ingresar'}
          </button>
        </form>
      </section>
    ) : (
      <section className="workspace" aria-busy={busy}>
        <article className="panel identity-panel">
```

```
          <h2>Identidad autenticada</h2>
          {user ? (
            <dl>
              <div><dt>Nombre</dt><dd>{user.name}</dd></div>
              <div><dt>Correo</dt><dd>{user.email}</dd></div>
              <div><dt>Rol</dt><dd>{user.role}</dd></div>
            </dl>
          ) : <p>Validando el token...</p>}
        </article>
        <article className="panel">
          <div className="section-heading">
            <h2>Tareas recuperadas</h2>
            <button
              onClick={() => token && void loadProtectedData(token)}
              disabled={busy}
            >
              Actualizar
            </button>
          </div>
          {tasks.length === 0 ? (
            <p>No hay tareas para mostrar.</p>
          ) : (
            <ul className="task-list">
              {tasks.map((task) => (
                <li key={task.id}>
                  <span>{task.title}</span>
                  <strong>{task.status}</strong>
                </li>
              ))}
            </ul>
          )}
        </article>
      </section>
    )}

    <p className="status-message" role="status">{message}</p>
  </main>
  );
}
```

Partes importantes.

- El navegador nunca envía userId ni role para identificarse.
- Promise.all reduce espera porque /me y /tasks son independientes.
- Cerrar sesión elimina el token local; no invalida tokens en el servidor porque no existe una lista de revocación.

Resultado esperado. El usuario puede iniciar sesión, ver su perfil y consultar tareas protegidas.

Posibles errores y corrección.

- Si el formulario devuelve 401, confirma que la cuenta exista y usa su contraseña real.
- Si al recargar vuelve al login, revisa sessionStorage y que el token no haya expirado.
- Si React ejecuta el efecto dos veces en desarrollo, recuerda que StrictMode ayuda a detectar efectos frágiles.

Paso 18. Montar React y agregar estilos adaptables

Objetivo. Completar una interfaz legible sin mezclar presentación con solicitudes.

main.tsx monta App dentro de StrictMode. styles.css organiza cabecera, formulario, identidad y lista con una adaptación para pantallas estrechas. El CSS no contiene datos ni reglas del negocio.

Ruta. client/src/main.tsx y client/src/styles.css

Partes importantes.

- role=status anuncia cambios del mensaje a tecnologías de asistencia.
- Los labels están asociados a sus inputs.
- La lista usa task.id como key estable.

Resultado esperado. Vite puede renderizar el cliente sin advertencias de TypeScript.

Posibles errores y corrección.

- Si la pantalla queda vacía, revisa la consola y el elemento root.
- Si faltan estilos, confirma la importación ./styles.css en main.tsx.
- No uses el índice del arreglo como key si ya existe un id.

client/src/main.tsx

Código completo

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.js';
import './styles.css';

const root = document.getElementById('root');
if (!root) throw new Error('No se encontró el elemento raíz de React.');

createRoot(root).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

client/src/styles.css

Código completo

```
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #172033;
  background: #eef2f7;
  font-synthesis: none;
}

* { box-sizing: border-box; }

body { margin: 0; min-width: 320px; min-height: 100vh; }

button, input { font: inherit; }

button {
  border: 0;
  border-radius: 0.55rem;
  padding: 0.7rem 1rem;
  color: #fff;
  background: #1f4f7a;
  cursor: pointer;
}

button:disabled { cursor: wait; opacity: 0.65; }

.page-shell { width: min(960px, calc(100% - 2rem)); margin: 0 auto; padding: 2rem 0 3rem; }

.page-header, .section-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}

.page-header h1 { margin: 0.2rem 0; font-size: clamp(2rem, 5vw, 3rem); }

.page-header p { margin: 0; color: #526173; }

.eyebrow { font-size: 0.78rem; font-weight: 800; letter-spacing: 0.11em; text-transform: uppercase; }

.panel {
  margin-top: 1.5rem;
  padding: 1.5rem;
  border: 1px solid #d4dde8;
  border-radius: 0.85rem;
  background: #fff;
  box-shadow: 0 10px 28px rgba(31, 79, 122, 0.08);
}
```

```
.panel h2 { margin-top: 0; }

.login-panel { max-width: 520px; }

form { display: grid; gap: 0.7rem; }

label { font-weight: 700; }

input {
  width: 100%;
  border: 1px solid #aebccb;
  border-radius: 0.5rem;
  padding: 0.75rem;
}

input:focus { outline: 3px solid #c5def2; border-color: #1f4f7a; }

.workspace { display: grid; grid-template-columns: minmax(220px, 0.8fr) minmax(0, 1.6fr); gap: 1rem; }

dl { margin: 0; }

dl div { padding: 0.65rem 0; border-bottom: 1px solid #e5ebf2; }

dt { font-size: 0.8rem; color: #607084; }

dd { margin: 0.2rem 0 0; overflow-wrap: anywhere; }

.task-list { list-style: none; margin: 0; padding: 0; }

.task-list li {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  padding: 0.85rem 0;
  border-bottom: 1px solid #e5ebf2;
}

.task-list strong { color: #1f4f7a; font-size: 0.85rem; }

.status-message { min-height: 1.5rem; color: #455568; }

@media (max-width: 720px) {
  .page-header, .section-heading { align-items: flex-start; }
  .workspace { grid-template-columns: 1fr; }
}
```

Parte V Comprobación guiada

Paso 19. Ejecutar la verificación completa

Objetivo. Comprobar tipos, pruebas y builds con una sola orden.

verify ejecuta el proceso en un orden que facilita localizar el fallo. Las pruebas automatizadas no requieren Atlas, pero el build confirma que el código de producción se genera.

Ruta. Raíz del repositorio; se generan dist y client/dist, ambos ignorados.

Comando

```
pnpm verify
```

Resultado esperado. check termina, Vitest aprueba 6 pruebas, tsc genera dist y Vite genera client/dist.

Posibles errores y corrección.

- Lee la primera orden que falla; las posteriores no se ejecutan.
- Si Vitest queda en modo observación, usa pnpm test y no pnpm test:watch.
- Si Vite informa una versión de Node incompatible, instala Node 20.19 o superior.

Paso 20. Iniciar backend y cliente en terminales separadas

Objetivo. Probar la integración real con Atlas y el navegador.

La primera terminal conecta la API a MongoDB Atlas. La segunda inicia Vite en 5173. Debes crear client/.env y confirmar que el .env del backend permita exactamente ese origen.

Ruta. Terminal 1 en la raíz y Terminal 2 en la raíz mediante el script client:dev.

Comando

```
Terminal 1:
pnpm dev

Terminal 2:
pnpm client:dev
```

Resultado esperado. La API escucha en http://localhost:3000 y Vite muestra http://localhost:5173.

Posibles errores y corrección.

- Si Atlas no conecta, revisa Network Access, usuario, contraseña codificada y MONGODB_DB_NAME.
- Si el navegador muestra CORS, compara protocolo, host y puerto con CORS_ORIGIN.
- Si 3000 o 5173 está ocupado, detén el proceso que lo usa antes de cambiar configuración.

Paso 21. Probar la documentación interactiva

Objetivo. Confirmar que OpenAPI describe operaciones públicas y protegidas.

Abre Swagger UI, ejecuta health y login, copia data.accessToken, selecciona Authorize y pega únicamente el token. Después prueba /me, GET /api/tasks y una operación negativa.

Ruta. Navegador en http://localhost:3000/api-docs; no se modifica un archivo.

Comando

```
Abrir http://localhost:3000/api-docs
Abrir http://localhost:3000/api-docs.json
```

Resultado esperado. Swagger muestra tres grupos, permite autorizar y devuelve códigos coherentes con la especificación.

Posibles errores y corrección.

- Si escribes Bearer dentro del cuadro, Swagger enviará el esquema duplicado; pega solo el JWT.
- Si Try it out apunta a otra URL, revisa servers en openapi.ts.
- Si una respuesta real no coincide con la documentada, corrige el contrato o el código antes de entregar.

Paso 22. Comprobar Postman y el cliente React

Objetivo. Ejecutar pruebas positivas, negativas, de regresión y de navegador.

Conserva la colección A10 y agrega /me y documentación. Ejecuta primero la matriz siguiente en Postman con Atlas. Después abre React, inicia sesión y confirma que muestra la misma identidad y las tareas.

Ruta. Postman, navegador y colección exportada en evidencias; no se modifica código.

Resultado esperado. Postman valida el sistema real y React demuestra que CORS y Bearer funcionan en navegador.

Posibles errores y corrección.

- Postman no aplica CORS; la evidencia de CORS debe incluir el navegador o el encabezado Origin.
- No captures el token completo. Ocúltalo antes de entregar evidencias.
- Si React muestra una cuenta distinta, limpia sessionStorage e inicia sesión otra vez.

9 Matriz de pruebas guiadas

Tipo	Método y ruta	Datos o autorización	Código	Resultado
Positiva	GET /health	Sin token	200	data.status = up
Documentación	GET /api-docs.json	Sin token	200	openapi = 3.1.0
Positiva	POST /api/auth/login	Correo y contraseña	200	Devuelve accessToken
Negativa	GET /api/auth/me	Sin token	401	AUTHENTICATION_REQUIRED
Positiva	GET /api/auth/me	Bearer user	200	Devuelve usuario sin hash
Negativa	GET /api/tasks	Sin token	401	Contrato uniforme
Regresión	GET /api/tasks?...	Bearer user y filtros	200	data y meta correctos
Positiva	POST /api/tasks	Bearer user y title	201	Tarea persistida
Autorización	DELETE /api/tasks/:id	Bearer user	403	FORBIDDEN
Autorización	DELETE /api/tasks/:id	Bearer admin	204	Sin cuerpo
Negativa	GET /api/tasks?page=0	Bearer válido	422	QUERY_VALIDATION_ERROR
Integración	React login y carga	Cuenta válida	200	Identidad y tareas visibles

10 Pruebas de Postman

En la solicitud Login agrega el siguiente script en Tests. Guarda el token únicamente en el entorno local de Postman. No exportes el valor actual de la variable.

Script de prueba para Login

```
pm.test("Login correcto", function () {
  pm.response.to.have.status(200);
});

const body = pm.response.json();
pm.expect(body.data.tokenType).to.eql("Bearer");
pm.environment.set("userToken", body.data.accessToken);
```

En GET /api/auth/me configura Bearer {{userToken}} y agrega:

Script de prueba para Me

```
pm.test("Identidad autenticada", function () {
  pm.response.to.have.status(200);
  const body = pm.response.json();
  pm.expect(body.data.user).to.have.property("id");
  pm.expect(body.data.user).to.have.property("role");
  pm.expect(body.data.user).not.to.have.property("passwordHash");
});
```

En GET /api/tasks?page=1&limit=10 agrega:

Script de regresión para Tareas

```
pm.test("Lista paginada protegida", function () {
  pm.response.to.have.status(200);
  const body = pm.response.json();
  pm.expect(body.data).to.be.an("array");
  pm.expect(body.meta.page).to.eql(1);
  pm.expect(body.meta.limit).to.eql(10);
});
```

Duplica /me, selecciona No Auth y confirma 401. Duplica una consulta de tareas, envía page=0 y confirma 422. Conserva una carpeta A11 con resultados, pero borra tokens y contraseñas antes de exportar la colección.

11 Registro de evidencias en Markdown

Crea el archivo siguiente y sustituye cada marcador. Usa nombres de capturas, no pegues tokens, secretos, contraseñas ni la URI de Atlas.

Ruta evidencias/ec3-a11.md

```
# Evidencias EC3 F2 A11

## Datos generales
- Nombre:
- Grupo:
- Repositorio y rama:
- Commit final:

## Ambiente
- Versión de Node:
- Versión de PNPM:
- URL local del backend: http://localhost:3000
- URL local del cliente: http://localhost:5173
- Base de datos usada: [nombre sin credenciales]

## Verificación automatizada
- Resultado de `pnpm verify`:
- Pruebas aprobadas:
- Evidencia: [captura sin secretos]

## Documentación
- `/api-docs.json` devuelve OpenAPI 3.1.0: [sí/no]
- Operaciones probadas desde Swagger:
- Diferencia encontrada entre código y documentación: [ninguna o explicación]

## Postman con Atlas
- Cuenta: [correo parcialmente oculto]
- Me con token: [resultado]
- Me sin token: [resultado]
- Lista paginada: [resultado]
- DELETE user/admin: [resultados]

## Integración React
- Login correcto: [sí/no]
- Identidad visible: [sí/no]
- Tareas visibles: [sí/no]
- CORS comprobado: [explicación]
- Evidencia: [captura con token oculto]

## Regresión
- Health:
- Errores uniformes y requestId:
- CRUD persistente:
- Paginación, filtros, búsqueda y ordenamiento:

## Conclusión
[Explica qué comprueban las pruebas automáticas, qué comprueba Atlas y qué comprueba el navegador.]
```

12 Desafío individual

Amplía el cliente React para crear una tarea mediante POST /api/tasks y actualizar la lista sin recargar la página. Debes reutilizar apiRequest, validar que el título no esté vacío, mostrar el error uniforme y conservar el token fuera de props y de la URL.

Criterio	Condición de cumplimiento
Formulario	Campo title y botón con estado de carga.
Solicitud	POST /api/tasks con JSON y Bearer.
Actualización	La tarea aparece después de una respuesta 201.
Error	Se muestra message y requestId cuando existan.
Prueba	Nuevo caso automatizado o comprobación Postman documentada.
Calidad	Sin any, sin token en URL o consola y pnpm verify correcto.

13 Preguntas de cierre

Responde en evidencias/ec3-a11-reflexion.md con tus propias palabras:

1. ¿Qué parte de la aplicación se reemplaza con dobles en api.test.ts y qué parte sigue siendo real?
2. ¿Por qué una suite aprobada no demuestra que MongoDB Atlas esté configurado correctamente?
3. ¿Qué diferencia existe entre OpenAPI y Swagger UI?
4. ¿Por qué GET /api/auth/me obtiene el id desde el token y no desde params o body?
5. ¿Por qué authRateLimiter ya no se monta en todo authRouter?
6. ¿Qué diferencia hay entre un error 401 y uno 403 en el cliente?
7. ¿Por qué Postman puede funcionar aunque el navegador informe CORS?
8. ¿Qué riesgo conserva sessionStorage y por qué se usa solo como solución didáctica?
9. ¿Qué contrato comparten openapi.ts, types.ts y las respuestas reales?
10. ¿Qué prueba agregarías primero antes de desplegar la actividad 12 y por qué?

14 Evidencias de entrega

- Repositorio task-api-express en la rama solicitada.
- Código del backend, pruebas, OpenAPI y carpeta client; sin node_modules, dist ni .env.
- pnpm-lock.yaml actualizado para los dos paquetes.
- Archivo evidencias/ec3-a11.md completo.
- Archivo evidencias/ec3-a11-reflexion.md con respuestas propias.
- Colección Postman exportada sin valores actuales de tokens o contraseñas.
- Capturas de pnpm verify, Swagger UI, prueba negativa, React y regresión con Atlas.
- Historial de commits y estado final sin secretos.

15 Lista de verificación

Verificación	Sí/No
Conservé la arquitectura, JWT, roles y Mongoose 8.24.4 de A10.	
El limitador se aplica solo a register y login.	
GET /api/auth/me exige JWT y no expone passwordHash.	
OpenAPI documenta rutas, códigos, esquemas y bearerAuth.	
Swagger UI y /api-docs.json responden correctamente.	
Las pruebas usan dobles de servicio y tokens JWT reales.	
pnpm test aprueba los seis casos.	
El cliente usa VITE_API_URL sin secretos.	
React envía Authorization Bearer y maneja errores uniformes.	
Probé manualmente Atlas, Postman, Swagger y navegador.	
pnpm verify termina sin errores.	
No entregué .env, tokens, contraseñas, URI, node_modules ni dist.	

16 Problemas frecuentes y soluciones

Problema	Causa probable	Solución
Swagger queda en blanco	CSP global bloquea recursos	Usa documentationSecurityHeaders solo en /api-docs.
Swagger envía Bearer Bearer	Se pegó el esquema con el token	En Authorize pega solo el JWT.
Vitest no encuentra variables	setupFiles no cargó	Revisa vitest.config.ts y tests/setup-env.ts.
Pruebas tocan Atlas	Se inició server o database	Importa app y simula servicios; no llames connectDatabase.
React muestra Failed to fetch	API apagada, URL o CORS	Revisa ambas terminales, VITE_API_URL y CORS_ORIGIN.
React recibe 401	Token ausente, vencido o inválido	Limpia sessionStorage e inicia sesión otra vez.
React recibe HTML	URL apunta al cliente	VITE_API_URL debe ser http://localhost:3000.
Vite usa 5174	5173 está ocupado	Detén el proceso; strictPort evita un cambio oculto.
pnpm install ignora client	No está en workspace	Agrega client en pnpm-workspace.yaml.
Build falla por .js	Importación relativa incompleta	Conserva extensiones .js en backend y archivos TS/TSX.
Atlas falla pero test aprueba	Los dobles aíslan persistencia	Realiza la matriz Postman con el clúster real.
Token aparece en evidencia	Captura o exportación insegura	Revoca o deja expirar y reemplaza la evidencia ocultándolo.

17 Propuesta de commits

Secuencia sugerida

```
git add package.json pnpm-lock.yaml pnpm-workspace.yaml tsconfig.test.json vitest.config.ts
git commit -m "test: configura pruebas automatizadas de la API"

git add src/repositories/user.repository.ts src/services/auth.service.ts src/controllers/auth.controller.ts src/routes/auth.routes.ts
git commit -m "feat: agrega consulta de identidad autenticada"

git add src/docs src/app.ts src/middlewares/security.middleware.ts
git commit -m "docs: publica contrato OpenAPI y Swagger UI"

git add tests
git commit -m "test: cubre autenticacion autorizacion y regresion"

git add client
git commit -m "feat: integra cliente React con la API protegida"

git add evidencias
git commit -m "docs: registra evidencias de la actividad 11"
```

Antes de confirmar, ejecuta git diff --cached y git status --ignored. Comprueba que .env, client/.env, node_modules, dist, tokens y capturas sensibles no estén preparados.

18 Recursos oficiales de consulta

Express manejo de errores: https://expressjs.com/en/guide/error-handling.html
Vitest guía: https://vitest.dev/guide/
SuperTest: https://github.com/forwardemail/supertest
Especificación OpenAPI 3.1: https://spec.openapis.org/oas/v3.1.0.html
Swagger UI: https://swagger.io/tools/swagger-ui/
React aprender React: https://react.dev/learn
Vite guía: https://vite.dev/guide/
MDN Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
MDN sessionStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage
MDN CORS: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

19 Verificación técnica de esta guía

El proyecto de referencia fue reconstruido sobre A10 con las versiones exactas mostradas. Se ejecutó pnpm verify. TypeScript validó el backend y las pruebas en modo estricto, Vitest aprobó un archivo con seis casos, tsc generó dist y Vite produjo el build del cliente.

La suite comprobó health, cabeceras, requestId, JSON OpenAPI, Swagger UI, registro, login, /me, token ausente, consulta validada, creación, 403 de user, 204 de admin, 422 uniforme y CORS permitido o no permitido. Los servicios de negocio se sustituyeron con dobles; token.service, middleware, rutas, controladores y respuestas de Express permanecieron reales.

No se usaron credenciales reales de MongoDB Atlas ni una sesión gráfica del navegador durante la comprobación automatizada. Por ello la validación de persistencia y CORS real corresponde a los pasos 20 a 22 con el ambiente del estudiante. La guía no afirma que un clúster particular funcione hasta realizar esas pruebas.

20 Cierre de la actividad

Ahora la API no solo funciona: también puede comprobarse, explicarse y consumirse. Las pruebas automatizadas protegen el contrato HTTP, OpenAPI convierte ese contrato en documentación navegable y React demuestra el flujo real de autenticación desde un navegador.

Antes de entregar, ejecuta pnpm verify, recorre la matriz con Atlas, revisa Swagger, limpia las variables exportadas de Postman y busca secretos en Git. En EC3 F3 A12 completarás la integración, prepararás la configuración de producción, desplegarás y realizarás la liberación final.